



PREMIER UNIVERSITY CHITTAGONG

Department of Computer Science & Engineering REPORT

Course Code : CSE 4346
Course Title : Big Data Analytics Laboratory
Name of the Report : Retail-BigData-Analysis Using Hadoop
Date of submission : 15/09/2026

Submitted To:

MD Tamim Hossain
Lecture,
Department of Computer Science and
Engineering.
Premier University, Chittagong.

Submitted By:

NAME: Syed Nasibur Rahaman Ratul
ID: 0222220005101189

SECTION : C

SEMESTER: 8th

BATCH: 42nd

SESSION: Spring 2026

Remarks:

TABLE OF CONTENTS

Introduction	1
Dataset Description	2
MapReduce Analysis	3
1. City-wise Total Sales Analysis	
2. Product Category-wise Sales Analysis	
3. Customer-wise Purchase/Sales Analysis	
Pig Analysis	13
1. City-wise Sales Analysis	
2. Customer Category-wise Sales Analysis	
3. Payment Method-wise Transaction Analysis	
4. Monthly Sales Trend Analysis	
5. Top Products Analysis	
Conclusion	22

Introduction:

The project focuses on analyzing a large-scale retail transaction dataset using Apache Pig and Hadoop MapReduce within the Hadoop ecosystem. The dataset contains approximately 1 million retail transaction records, representing customer purchases across different cities, stores, payment methods, customer categories, seasons, and promotional activities.

At the core of this system is Hadoop, which provides a distributed framework for storing and processing large volumes of data. Hadoop mainly consists of two important components used in this project:

HDFS (Hadoop Distributed File System)

HDFS is responsible for storing the large retail transaction dataset. Instead of processing the complete dataset as a traditional local file, the dataset is stored in HDFS so that it can be accessed efficiently by Hadoop processing frameworks.

The dataset used in this project, `Retail_Transactions_Dataset.csv`, contains more than one million transaction records and is stored in HDFS before performing the analysis.

MapReduce Framework

MapReduce is the processing framework used to analyze the large dataset. It divides the processing work into two main phases: Mapper and Reducer. The Mapper processes individual transaction records and generates intermediate key-value pairs. The Reducer then groups and aggregates these values to produce meaningful analytical results.

Apache Pig

Apache Pig provides a high-level data analysis platform using Pig Latin. It simplifies large-scale data processing by providing operations such as loading, filtering, grouping, sorting, aggregation, and storing results. The project performs several analytical operations on the retail dataset. MapReduce is used for city-wise sales, product category-wise sales, and customer-wise purchase analysis. Apache Pig is used for city-wise sales, customer category-wise sales, payment method analysis, monthly sales trends, and top product analysis.

These analyses help identify sales distribution, customer behavior, payment preferences, temporal sales patterns, and frequently purchased products.

Dataset Description:

The dataset used in this project is the Retail Transactions Dataset, containing approximately 1,000,000 transaction records. Each record represents a retail transaction and contains information about the transaction, customer, products, total cost, payment method, city, store type, customer category, season, and promotion.

The dataset is suitable for Big Data analysis because of its large volume and variety of attributes.

Dataset Schema

Field Name	Data Type	Description
Transaction_ID	Integer	Unique identifier for each transaction
Date	String/Date	Date and time of the transaction
Customer_Name	String	Name of the customer
Product	String	Product or list of products purchased
Total_Items	Integer	Number of items in the transaction
Total_Cost	Double	Total cost of the transaction
Payment_Method	String	Payment method used
City	String	City where the transaction occurred
Store_Type	String	Type of retail store
Discount_Applied	Boolean	Indicates whether a discount was applied
Customer_Category	String	Category of the customer
Season	String	Season of the transaction
Promotion	String	Promotion applied to the transaction

Key Characteristics of the Dataset

- Large-scale: Contains approximately 1 million transaction records.
- Customer diversity: Contains different customer categories.
- Product diversity: Transactions contain different products and product combinations.
- Geographical information: Transactions are distributed across multiple cities.
- Payment diversity: Multiple payment methods are available.
- Sales information: Total_Cost can be used for revenue and sales analysis.
- Temporal information: The Date field supports monthly sales analysis.
- Store information: Different types of retail stores are represented.
- Promotion information: The dataset contains different promotional activities.

MapReduce Analysis:

1. City-wise Total Sales Analysis

Objective:

The objective of this analysis is to calculate the total sales generated in each city. The City field is used as the grouping key, while Total_Cost is aggregated to calculate the total sales for each city.

This analysis comprises of:

1. **Driver Class:** Configures the MapReduce job, Mapper, Reducer, input path, output path, and output data types.
2. **Mapper Class:** Extracts the city and total transaction cost and emits the city as the key and total cost as the value.
3. **Reducer Class:** Receives all transaction costs for each city and calculates their total.

Code Snippet:

```
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class Task1_CitySales {

    public static class Map
        extends Mapper<LongWritable, Text, Text, DoubleWritable> {

        private Text outKey = new Text();

        public void map(LongWritable key, Text value, Context context)
            throws IOException, InterruptedException {

            String line = value.toString();

            if (line.startsWith("Transaction_ID"))
                return;

            String[] f = line.split(
                ",(?=(?:[^\"]*" + "\"[^\"]*" + "\")*[^\"]*$)", -1
            );

            if (f.length < 13)
```

```

        return;

    try {
        String city = f[7].trim();
        double totalCost = Double.parseDouble(f[5].trim());

        outKey.set(city);
        context.write(outKey,
            new DoubleWritable(totalCost));

    } catch (Exception e) {
        // Ignore invalid records
    }
}

}

public static class Reduce
    extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {

    public void reduce(Text key,
        Iterable<DoubleWritable> values,
        Context context)
        throws IOException, InterruptedException {

        double total = 0;

        for (DoubleWritable value : values) {
            total += value.get();
        }

        context.write(key, new DoubleWritable(total));
    }
}

public static void main(String[] args) throws Exception {

    Job job = Job.getInstance(
        new Configuration(),
        "City Wise Total Sales"
    );

    job.setJarByClass(Task1_CitySales.class);

    job.setMapperClass(Map.class);
    job.setReducerClass(Reduce.class);

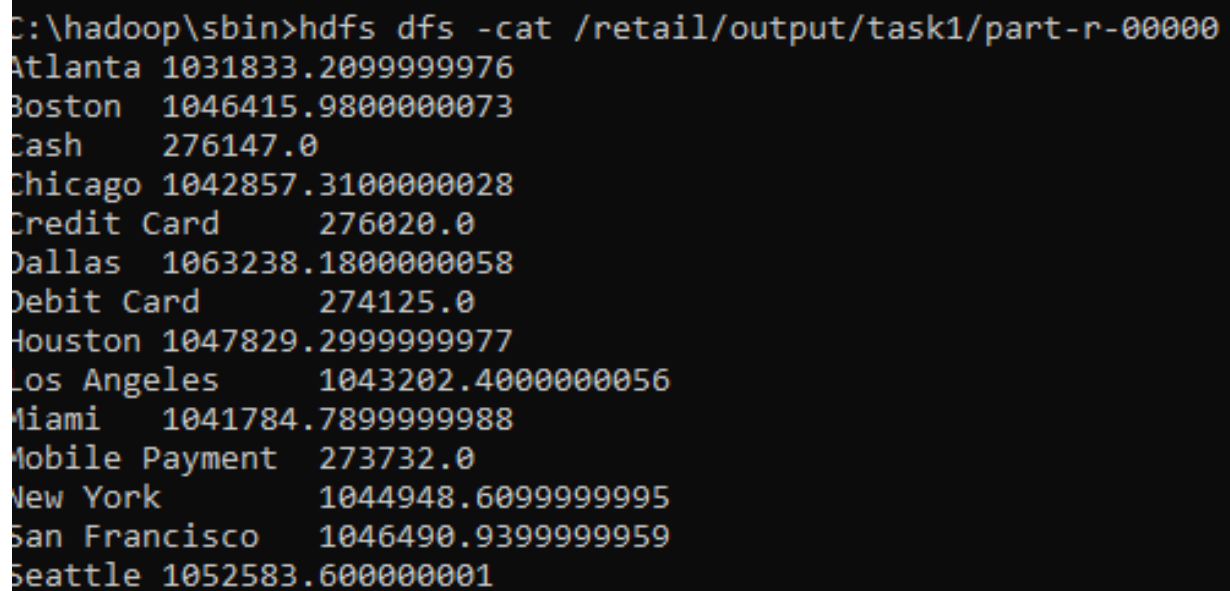
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(DoubleWritable.class);

    FileInputFormat.addInputPath(
        job, new Path(args[0])
    );
}

```

```
);  
  
FileOutputFormat.setOutputPath(  
    job, new Path(args[1])  
);  
  
System.exit(  
    job.waitForCompletion(true) ? 0 : 1  
);  
}  
}
```

Output Screenshot:



A screenshot of a terminal window with a black background and white text. The command executed is `C:\hadoop\sbin>hdfs dfs -cat /retail/output/task1/part-r-00000`. The output lists 14 categories with their corresponding values, formatted as `Category Value`.

Category	Value
Atlanta	1031833.2099999976
Boston	1046415.9800000073
Cash	276147.0
Chicago	1042857.3100000028
Credit Card	276020.0
Dallas	1063238.1800000058
Debit Card	274125.0
Houston	1047829.2999999977
Los Angeles	1043202.4000000056
Miami	1041784.7899999988
Mobile Payment	273732.0
New York	1044948.6099999995
San Francisco	1046490.9399999959
Seattle	1052583.600000001

2. Product Category Sales Analysis

Objective:

The objective of this analysis is to calculate the total sales associated with different product categories. Since the original dataset does not contain a separate `Product_Category` field, product information is processed to derive the relevant category.

This analysis comprises of:

1. **Driver Class:** Configures the MapReduce job and defines the Mapper and Reducer classes.
2. **Mapper Class:** Extracts the product information and determines the corresponding product category along with the transaction cost.
3. **Reducer Class:** Aggregates the transaction costs for each product category.

Code Snippet:

```
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class ProductCategorySales {

    public static class Map
        extends Mapper<LongWritable, Text, Text, DoubleWritable> {

        private Text outKey = new Text();

        public void map(LongWritable key, Text value, Context context)
            throws IOException, InterruptedException {

            String line = value.toString();

            if (line.startsWith("Transaction_ID"))
                return;

            String[] f = line.split(
                ",(?=(?:[^\"]*" + "\"[^\"]*" + "\")*" + "[^\"]*" + "$)", -1
            );

            if (f.length < 13)
                return;

            try {
```



```

String product = f[3].toLowerCase();
double cost = Double.parseDouble(f[5]);

String category;

if (product.contains("milk")
    || product.contains("bread")
    || product.contains("cheese")
    || product.contains("yogurt")
    || product.contains("ice cream")) {

    category = "Food";

} else if (product.contains("soap")
    || product.contains("shaving")
    || product.contains("toothpaste")
    || product.contains("tissues")) {

    category = "Personal Care";

} else if (product.contains("bulbs")
    || product.contains("hose")
    || product.contains("cans")) {

    category = "Household";

} else {
    category = "Other";
}

outKey.set(category);
context.write(
    outKey,
    new DoubleWritable(cost)
);

} catch (Exception e) {
    // Ignore invalid records
}
}

public static class Reduce
    extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {

    public void reduce(Text key,
        Iterable<DoubleWritable> values,
        Context context)
        throws IOException, InterruptedException {

        double total = 0;

```

```

        for (DoubleWritable value : values) {
            total += value.get();
        }

        context.write(
            key,
            new DoubleWritable(total)
        );
    }
}

public static void main(String[] args) throws Exception {

    Job job = Job.getInstance(
        new Configuration(),
        "Product Category Sales"
    );

    job.setJarByClass(ProductCategorySales.class);

    job.setMapperClass(Map.class);
    job.setReducerClass(Reduce.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(DoubleWritable.class);

    FileInputFormat.addInputPath(
        job, new Path(args[0])
    );

    FileOutputFormat.setOutputPath(
        job, new Path(args[1])
    );

    System.exit(
        job.waitForCompletion(true) ? 0 : 1
    );
}
}

```

Output Screenshot:

```

C:\RetailProject\MapReduce>hdfs dfs -cat /retail/output/product_category_sales/part-r-00000
Electronics      1168081.650000004
Food & Grocery   1.201437886000012E7
Other            2.9755168109999474E7
Personal Care    9517591.780000024

```

3. Customer-wise Purchase/Sales Analysis

Objective:

The objective of this analysis is to calculate the total purchase amount for each customer. The customer name is used as the grouping key and the total transaction cost is aggregated for each customer.

This analysis comprises of:

1. **Driver Class:** Configures the MapReduce job and specifies input/output paths.
2. **Mapper Class:** Emits Customer_Name as the key and Total_Cost as the value.
3. **Reducer Class:** Sums all transaction costs associated with each customer.

Code Snippet:

```
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class CustomerSales {

    public static class Map
        extends Mapper<LongWritable, Text, Text, DoubleWritable> {

        private Text outKey = new Text();

        public void map(LongWritable key, Text value, Context context)
            throws IOException, InterruptedException {

            String line = value.toString();

            if (line.startsWith("Transaction_ID"))
                return;

            String[] f = line.split(
                ",(?=(?:[^\"]*" + "[^\\"]*" + "\\")*\"[^\"]*$)", -1
            );

            if (f.length < 13)
                return;

            try {
                String customer = f[2].trim();
```

```

        double cost = Double.parseDouble(f[5].trim());

        outKey.set(customer);

        context.write(
            outKey,
            new DoubleWritable(cost)
        );

    } catch (Exception e) {
        // Ignore invalid records
    }
}

}

public static class Reduce
    extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {

    public void reduce(Text key,
        Iterable<DoubleWritable> values,
        Context context)
        throws IOException, InterruptedException {

        double total = 0;

        for (DoubleWritable value : values) {
            total += value.get();
        }

        context.write(
            key,
            new DoubleWritable(total)
        );
    }
}

public static void main(String[] args) throws Exception {

    Job job = Job.getInstance(
        new Configuration(),
        "Customer Wise Sales"
    );

    job.setJarByClass(CustomerSales.class);

    job.setMapperClass(Map.class);
    job.setReducerClass(Reduce.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(DoubleWritable.class);

```

```

FileInputFormat.addInputPath(
    job, new Path(args[0])
);

FileOutputFormat.setOutputPath(
    job, new Path(args[1])
);

System.exit(
    job.waitForCompletion(true) ? 0 : 1
);
}
}

```

Output Screenshot:

```

Victor Hammond 38.37
Victor Hammond DDS 5.65
Victor Hampton 34.31
Victor Hansen 96.39
Victor Harper 70.93
Victor Harrell 6.56
Victor Harrington 54.49
Victor Harris 208.68
Victor Harrison 70.6
Victor Hart 61.129999999999995
Victor Hartman 54.63
Victor Hayden 63.74
Victor Hayes 10.42
Victor Haynes 160.88
Victor Henderson 121.66
Victor Henry 134.91
Victor Henson 42.36
Victor Herman 60.59
Victor Hernandez 296.70000000000005
Victor Herrera 104.47
Victor Herring 42.39
Victor Hicks 17.35
Victor Hill 330.21000000000004
Victor Hines 182.25
Victor Hinton 25.17
Victor Ho 60.79
Victor Hobbs 53.78
Victor Hodges 94.39
Victor Hoffman 13.81
Victor Hoffman DVM 9.06
Victor Holden 69.32000000000001
Victor Holloway 250.76999999999998
Victor Holmes 221.06
Victor Holt 98.42999999999999
Victor Hooper 97.65
Victor Horn 74.15
Victor Horton 99.34
Victor Houston 72.81
Victor Howard 240.79
Victor Howell 182.01999999999998
Victor Huerta 42.5
Victor Hughes 134.36
Victor Hunt 27.35
Victor Hunter 246.06
Victor Huynh 81.1
Victor Jackson 406.07000000000005
Victor Jacobs 91.26
Victor James 16.59
Victor Jenkins 329.46000000000004

```

Pig Analysis:

1. City-wise Sales Analysis

This analysis calculates the total sales generated by each city. The transaction data is extracted from the raw CSV lines, grouped according to city, and the Total_Cost values are summed. The final results are sorted according to total sales.

Code Snippet:

```
raw = LOAD '/retail/input/Retail_Transactions_Dataset.csv'
USING TextLoader()
AS (line:chararray);

data = FILTER raw
BY NOT STARTSWITH(line, 'Transaction_ID');

parsed = FOREACH data GENERATE
    REGEX_EXTRACT(
        line,
        '^(\d+),(.?),(^[,]+),"(.*?)"',(\d+),([0-9.]+),([,^,]+),([,^,]+)',
        8
    ) AS city,
    (double)REGEX_EXTRACT(
        line,
        '^(\d+),(.?),(^[,]+),"(.*?)"',(\d+),([0-9.]+)',
        6
    ) AS total_cost;

valid = FILTER parsed
BY city IS NOT NULL AND total_cost IS NOT NULL;

grp = GROUP valid BY city;

result = FOREACH grp GENERATE
    group AS city,
    SUM(valid.total_cost) AS total_sales;

ordered = ORDER result
BY total_sales DESC;

STORE ordered
INTO '/retail/pig/city_sales'
USING PigStorage(',');
```

Output Screenshot:

```
C:\RetailProject>hdfs dfs -cat /retail/pig/city_sales/part-r-00000
Chicago,4220330.14000001
Boston,4216891.980000005
Dallas,4213873.350000026
New York,4207521.310000003
Houston,4199225.479999991
Miami,4198713.650000004
San Francisco,4194608.920000044
Los Angeles,4189190.790000006
Seattle,4182781.8299999954
Atlanta,4170898.6299999966
```

2. Customer Category-wise Sales Analysis

This analysis groups transactions according to the Customer_Category field. It calculates the total sales generated by each customer category and sorts the categories based on their total sales.

Code Snippet:

```
raw = LOAD '/retail/input/Retail_Transactions_Dataset.csv'
      USING TextLoader()
      AS (line:chararray);

data = FILTER raw
      BY NOT STARTSWITH(line, 'Transaction_ID');

parsed = FOREACH data GENERATE
          REGEX_EXTRACT(
              line,
              '^(\d+),(.*),([\d+]),"(.*)"',(\d+),([0-9.]+),([\d+]),([\d+]),([\d+]),([\d+]),([\d+]),
              11
          ) AS category,
          (double)REGEX_EXTRACT(
              line,
              '^(\d+),(.*),([\d+]),"(.*)"',(\d+),([0-9.]+)',
              6
          ) AS total_cost;

valid = FILTER parsed
      BY category IS NOT NULL AND total_cost IS NOT NULL;

grp = GROUP valid BY category;

result = FOREACH grp GENERATE
```

```

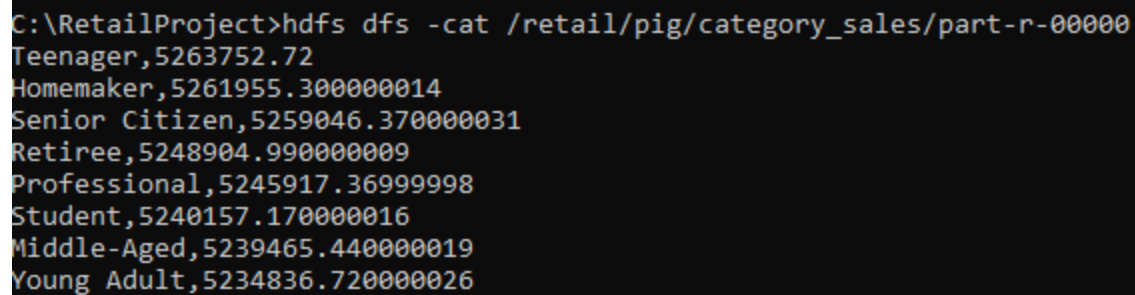
group AS customer_category,
SUM(valid.total_cost) AS total_sales;

ordered = ORDER result
BY total_sales DESC;

STORE ordered
INTO '/retail/pig/category_sales'
USING PigStorage(',');

```

Output Screenshot:



```

C:\RetailProject>hdfs dfs -cat /retail/pig/category_sales/part-r-00000
Teenager,5263752.72
Homemaker,5261955.300000014
Senior Citizen,5259046.370000031
Retiree,5248904.990000009
Professional,5245917.36999998
Student,5240157.170000016
Middle-Aged,5239465.440000019
Young Adult,5234836.720000026

```

3. Payment Method-wise Transaction Analysis

This analysis examines customer payment behavior by grouping transactions according to payment method. It calculates both the number of transactions and total sales associated with each payment method.

Code Snippet:

```

raw = LOAD '/retail/input/Retail_Transactions_Dataset.csv'
USING TextLoader()
AS (line:chararray);

data = FILTER raw
BY NOT STARTSWITH(line, 'Transaction_ID');

parsed = FOREACH data GENERATE
  REGEX_EXTRACT(
    line,
    '^(\\d+),(. *?),([^,]+),"(.*?)"',(\\d+),([0-9.]+),([^,]+)',
    7
  ) AS payment_method,
  (double)REGEX_EXTRACT(
    line,
    '^(\\d+),(. *?),([^,]+),"(.*?)"',(\\d+),([0-9.]+)',

```



```
        6
    ) AS total_cost;

valid = FILTER parsed
BY payment_method IS NOT NULL
AND total_cost IS NOT NULL;

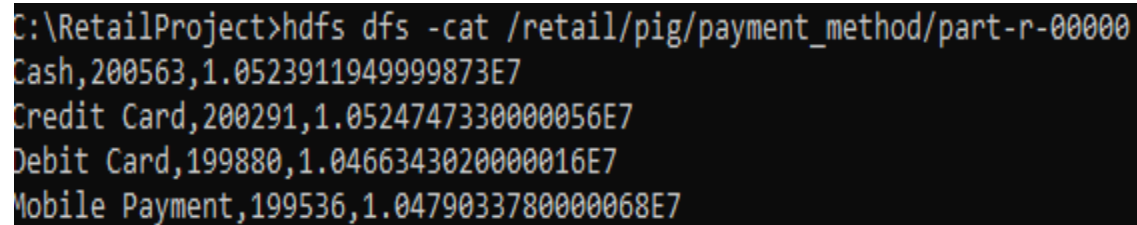
grp = GROUP valid BY payment_method;

result = FOREACH grp GENERATE
    group AS payment_method,
    COUNT(valid) AS transactions,
    SUM(valid.total_cost) AS total_sales;

ordered = ORDER result
BY transactions DESC;

STORE ordered
INTO '/retail/pig/payment_method'
USING PigStorage(',');
```

Output Screenshot:



```
C:\RetailProject>hdfs dfs -cat /retail/pig/payment_method/part-r-00000
Cash,200563,1.0523911949999873E7
Credit Card,200291,1.0524747330000056E7
Debit Card,199880,1.0466343020000016E7
Mobile Payment,199536,1.0479033780000068E7
```

4. Monthly Sales Trend Analysis

This analysis extracts the year and month from the transaction date and calculates the total sales for each month. The result helps identify changes and trends in sales over time.

Code Snippet:

```
raw = LOAD '/retail/input/Retail_Transactions_Dataset.csv'
USING TextLoader()
AS (line:chararray);
```

```
data = FILTER raw
BY NOT STARTSWITH(line, 'Transaction_ID');
```

```
parsed = FOREACH data GENERATE
    SUBSTRING(
        REGEX_EXTRACT(
            line,
            '^(\d+),([\^,]+)',
            2
        ),
        0,
        7
    ) AS month,

    (double)REGEX_EXTRACT(
        line,
        '^(\d+),([\^,]+),([\^,]+),\"(.*)\",(\d+),([0-9.]*)',
        6
    ) AS total_cost;
```

```
valid = FILTER parsed
BY month IS NOT NULL
AND total_cost IS NOT NULL;
```

```
grp = GROUP valid BY month;
```

```
result = FOREACH grp GENERATE
    group AS month,
    SUM(valid.total_cost) AS total_sales;
```

```
ordered = ORDER result
BY month ASC;
```

```
STORE ordered
INTO '/retail/pig/monthly_sales'
USING PigStorage(',');
```

Output Screenshot:

```
C:\RetailProject>hdfs dfs -cat /retail/pig/monthly_sales/part-r-00000
2020-01,818673.5099999995
2020-02,762716.7599999971
2020-03,812776.4500000017
2020-04,795672.0799999994
2020-05,797627.0299999983
2020-06,793480.1699999989
2020-07,809199.09
2020-08,816095.5699999982
2020-09,790916.4199999995
2020-10,812105.2200000035
2020-11,803482.2499999978
2020-12,811467.6099999964
2021-01,812189.4300000009
2021-02,735153.5600000005
2021-03,821228.0599999984
2021-04,782568.47
2021-05,813270.810000003
2021-06,788282.6800000014
2021-07,817026.9899999953
2021-08,805832.9699999985
2021-09,784507.6199999971
2021-10,814016.7900000003
2021-11,797115.9599999997
2021-12,813463.7200000007
2022-01,816925.1399999994
2022-02,727626.5000000034
2022-03,815663.4199999964
2022-04,792648.32
2022-05,819147.08
2022-06,803031.6600000006
2022-07,813176.8300000003
2022-08,803833.1800000011
2022-09,785521.5599999982
2022-10,809484.8499999959
2022-11,782922.2800000008
2022-12,798514.3799999999
2023-01,810388.4399999995
2023-02,741812.2000000032
2023-03,826800.7599999993
2023-04,784580.8299999979
2023-05,808722.7600000006
2023-06,772168.5799999993
2023-07,822788.1600000069
2023-08,814098.4300000012
2023-09,785134.7599999992
2023-10,814287.1400000004
2023-11,790209.96
2023-12,816451.8699999985
2024-01,813328.8600000016
2024-02,753311.1299999978
2024-03,811927.2099999988
2024-04,780846.9700000029
2024-05,469813.6000000001
```

5. Top Products Analysis

This analysis identifies the most frequently occurring products in the dataset. Since the Product field may contain multiple products in a single transaction, the product list is separated into individual products. The products are then grouped and counted, and the top 10 products are selected.

Code Snippet:

```
raw = LOAD '/retail/input/Retail_Transactions_Dataset.csv'
      USING TextLoader()
      AS (line:chararray);
```

```
data = FILTER raw
      BY NOT STARTSWITH(line, 'Transaction_ID');
```

```
parsed = FOREACH data GENERATE
          REGEX_EXTRACT(
              line,
              '^([\],]*,[\],]*,[\],*"?\\[(.*?)\\]"?;',
              1
          ) AS product_list;
```

```
valid = FILTER parsed
      BY product_list IS NOT NULL;
```

```
tokens = FOREACH valid GENERATE
          FLATTEN(TOKENIZE(product_list, ','))
          AS product;
```

```
clean_products = FOREACH tokens GENERATE
          TRIM(product) AS product;
```

```
valid_products = FILTER clean_products
      BY product IS NOT NULL
      AND product != '';
```

```
grp = GROUP valid_products BY product;
```

```
result = FOREACH grp GENERATE
          group AS product,
          COUNT(valid_products) AS transaction_count;
```

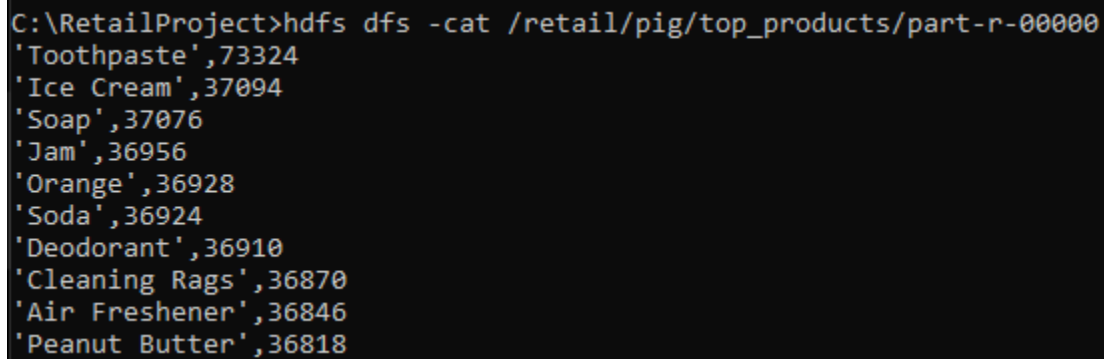
```
ordered = ORDER result
      BY transaction_count DESC;
```

```
top = LIMIT ordered 10;
```

```
STORE top
      INTO '/retail/pig/top_products'
```

USING PigStorage(',');

Output Screenshot:



```
C:\RetailProject>hdfs dfs -cat /retail/pig/top_products/part-r-00000
'Toothpaste',73324
'Ice Cream',37094
'Soap',37076
'Jam',36956
'Orange',36928
'Soda',36924
'Deodorant',36910
'Cleaning Rags',36870
'Air Freshener',36846
'Peanut Butter',36818
```

Conclusion:

The project successfully demonstrates the use of Hadoop MapReduce and Apache Pig for processing and analyzing a large-scale retail transaction dataset containing approximately one million records. The MapReduce implementation was used to perform City-wise Total Sales Analysis, Product Category-wise Sales Analysis, and Customer-wise Purchase/Sales Analysis. These analyses demonstrate how the Mapper and Reducer components can be used to process and aggregate large volumes of transaction data. Apache Pig was used to perform five additional analytical operations: City-wise Sales Analysis, Customer Category-wise Sales Analysis, Payment Method-wise Transaction Analysis, Monthly Sales Trend Analysis, and Top Products Analysis. These operations demonstrate the use of Pig Latin for loading, filtering, grouping, aggregation, sorting, and storing large-scale data. The project also demonstrates the difference between the two approaches. MapReduce requires detailed programming of Mapper, Reducer, and Driver classes, providing greater control over the processing logic. In contrast, Pig provides a higher-level and more concise approach for performing common data-processing operations. Overall, the project demonstrates how Hadoop, HDFS, MapReduce, and Apache Pig can be combined to process large-scale retail transaction data and extract useful business insights such as sales distribution, customer behavior, payment preferences, monthly trends, and popular products.